Java OOPs Concepts

Practice Questions & Solutions

Class | Object | Encapsulation | Inheritance | Polymorphism | Abstraction

1. Class, Object & Encapsulation

Q1. Class and Object – Create a class and access it using an object.

Solution:

```
class Student {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

public class ClassObjectDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Priti";
        s1.age = 21;
        s1.display();
    }
}
```

Output: Name: Priti, Age: 21

Q2. Encapsulation – Achieve data hiding using private fields and getter/setter methods.

Solution:

```
class Account {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        if (balance >= 0) {
            this.balance = balance;
        } else {
            System.out.println("Balance cannot be negative");
        }
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        Account acc = new Account();
        acc.setBalance(5000.0);
        System.out.println("Balance: " + acc.getBalance());

        acc.setBalance(-200);
    }
}
```

Output: Balance: 5000.0, then "Balance cannot be negative"

Q3. Constructor – Demonstrate default, parameterized, and constructor overloading.

Solution:

```
class Rectangle {
    int length, breadth;

    // Default constructor
    Rectangle() {
        length = 1;
        breadth = 1;
    }

    // Parameterized constructor (overloaded)
    Rectangle(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    int area() {
        return length * breadth;
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle(10, 5);

        System.out.println("r1 area: " + r1.area());
        System.out.println("r2 area: " + r2.area());
    }
}
```

Output: r1 area: 1, r2 area: 50

Q4. this Keyword – Use 'this' to resolve naming conflicts and chain constructors.

Solution:

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    Employee() {
        this("Unknown", 0.0);   // constructor chaining using this()
    }

    void show() {
```

```
        System.out.println(this.name + " earns " + this.salary);
    }
}

public class ThisKeywordDemo {
    public static void main(String[] args) {
        Employee e1 = new Employee("Rahul", 45000);
        Employee e2 = new Employee();

        e1.show();
        e2.show();
    }
}
```

Output: Rahul earns 45000.0, Unknown earns 0.0

2. Inheritance

Q5. Single Inheritance – A subclass inherits fields and methods from a parent class.

Solution:

```
class Animal {
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks.");
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output: This animal eats food. / The dog barks.

Q6. Multilevel Inheritance – Demonstrate a chain of inheritance (Grandparent → Parent → Child).

Solution:

```
class Vehicle {
    void start() {
        System.out.println("Vehicle started.");
    }
```

```
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is being driven.");
    }
}

class SportsCar extends Car {
    void turboBoost() {
        System.out.println("Turbo boost activated!");
    }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        SportsCar sc = new SportsCar();
        sc.start();
        sc.drive();
        sc.turboBoost();
    }
}
```

Output: Vehicle started. / Car is being driven. / Turbo boost activated!

Q7. super Keyword – Access parent class members and constructor from a subclass.

Solution:

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    void display() {
        System.out.println("Name: " + name);
    }
}

class Student extends Person {
    int rollNo;

    Student(String name, int rollNo) {
        super(name);            // calling parent constructor
        this.rollNo = rollNo;
    }

    void display() {
        super.display();        // calling parent method
        System.out.println("Roll No: " + rollNo);
    }
}
```

```
public class SuperKeywordDemo {
    public static void main(String[] args) {
        Student s = new Student("Priti", 101);
        s.display();
    }
}
```

Output: Name: Priti, Roll No: 101

3. Polymorphism

Q8. Compile-time Polymorphism – Method Overloading (same method name, different parameters).

Solution:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class MethodOverloadingDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));
        System.out.println(calc.add(2.5, 3.5));
        System.out.println(calc.add(1, 2, 3));
    }
}
```

Output: 5, 6.0, 6

Q9. Runtime Polymorphism – Method Overriding (subclass provides its own implementation).

Solution:

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape.");
    }
}

class Circle extends Shape {
    @Override
```

```
    void draw() {
        System.out.println("Drawing a circle.");
    }
}

class Square extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a square.");
    }
}

public class MethodOverridingDemo {
    public static void main(String[] args) {
        Shape s;

        s = new Circle();
        s.draw();          // runtime decides which draw() to call

        s = new Square();
        s.draw();
    }
}
```

Output: Drawing a circle. / Drawing a square.

4. Abstraction

Q10. Abstraction using Abstract Class – Hide implementation details, force subclasses to implement methods.

Solution:

```
abstract class Bank {
    abstract double interestRate();

    void welcome() {
        System.out.println("Welcome to the Bank.");
    }
}

class SBI extends Bank {
    double interestRate() {
        return 6.5;
    }
}

class HDFC extends Bank {
    double interestRate() {
        return 7.0;
    }
}
```

```
public class AbstractClassDemo {
    public static void main(String[] args) {
        Bank sbi = new SBI();
        Bank hdfc = new HDFC();

        sbi.welcome();
        System.out.println("SBI Interest Rate: " + sbi.interestRate());
        System.out.println("HDFC Interest Rate: " + hdfc.interestRate());
    }
}
```

Output: Welcome to the Bank. / SBI Interest Rate: 6.5 / HDFC Interest Rate: 7.0

Q11. Abstraction using Interface – Achieve 100% abstraction and multiple inheritance of type.

Solution:

```
interface Payable {
    void pay(double amount);
}

interface Refundable {
    void refund(double amount);
}

class Order implements Payable, Refundable {
    public void pay(double amount) {
        System.out.println("Paid amount: " + amount);
    }

    public void refund(double amount) {
        System.out.println("Refunded amount: " + amount);
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Order order = new Order();
        order.pay(1500.0);
        order.refund(500.0);
    }
}
```

Output: Paid amount: 1500.0 / Refunded amount: 500.0

5. Static and Final Keywords

Q12. static Keyword – Demonstrate static variable, static method, and static block.

Solution:

```
class Counter {
    static int count = 0;   // shared across all objects
```

```
    static {
        System.out.println("Static block executed.");
    }

    Counter() {
        count++;
    }

    static void showCount() {
        System.out.println("Total objects created: " + count);
    }
}

public class StaticKeywordDemo {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        new Counter();

        Counter.showCount();
    }
}
```

Output: Static block executed. / Total objects created: 3

Q13. final Keyword – Demonstrate final variable, final method, and final class.

Solution:

```
final class Constants {                 // final class cannot be extended
    static final double PI = 3.14159;   // final variable cannot change

    final void show() {                 // final method cannot be overridden
        System.out.println("Value of PI: " + PI);
    }
}

public class FinalKeywordDemo {
    public static void main(String[] args) {
        Constants c = new Constants();
        c.show();

        // PI = 3.14;   // Compile-time error: cannot assign a value to final variable
        // class Sub extends Constants {}  // Compile-time error: cannot inherit from
final class
    }
}
```

Output: Value of PI: 3.14159